

BNCmatmul 0.24 : 多倍長 基本線形計算・疎行列ルーチンの GPU 化と CPU 並列(OpenMP)同精度ルーチンとの性能比較レポート

device: NVIDIA H100 NVL (sm_90) x4 / CPU: 32-core AVX-512, OpenMP 32 threads / CUDA 12.x / 2026-07

1. 目的

gdtq(実数 double/single 系 EFT 多倍長: dd/td/qd, ds/ts/qs)および mpc_cuda(複素 MPC 多倍長)を用いて基本線形計算(AXPY, 行列×ベクトル, 行列×行列)と疎行列ベクトル積(SpMV)を GPU 実行し, 同一精度の CPU OpenMP 並列ルーチンと実行時間を比較した.

2. 手法

- GPU: H100 上で gdtq / mpc_cuda のデバイスカーネルを実行. 時間はカーネル実行のみ(データはデバイス常駐, cudaDeviceSynchronize で計測, reps 回の最小値).
- CPU: 同一精度の _bncomp_*(OpenMP, AVX-512, 32スレッド)ルーチン. SpMV の ds/ts/qs はライブラリに OMP 版が無いため直列(serial)を基準とした. 複素 mpc_cuda の CPU 基準は同一の mpc_cuda ホスト MPC 演算を OpenMP 並列化(同一精度・同一アルゴリズム).
- 速度向上率 speedup = (CPU-OMP 実行時間) / (GPU 実行時間). 正当性は relerr(GPU vs CPU の相対誤差最大値)で確認. MFLOPS は高位演算数(AXPY:2N, matvec:2N², matmul:2N³, SpMV:2nnz).

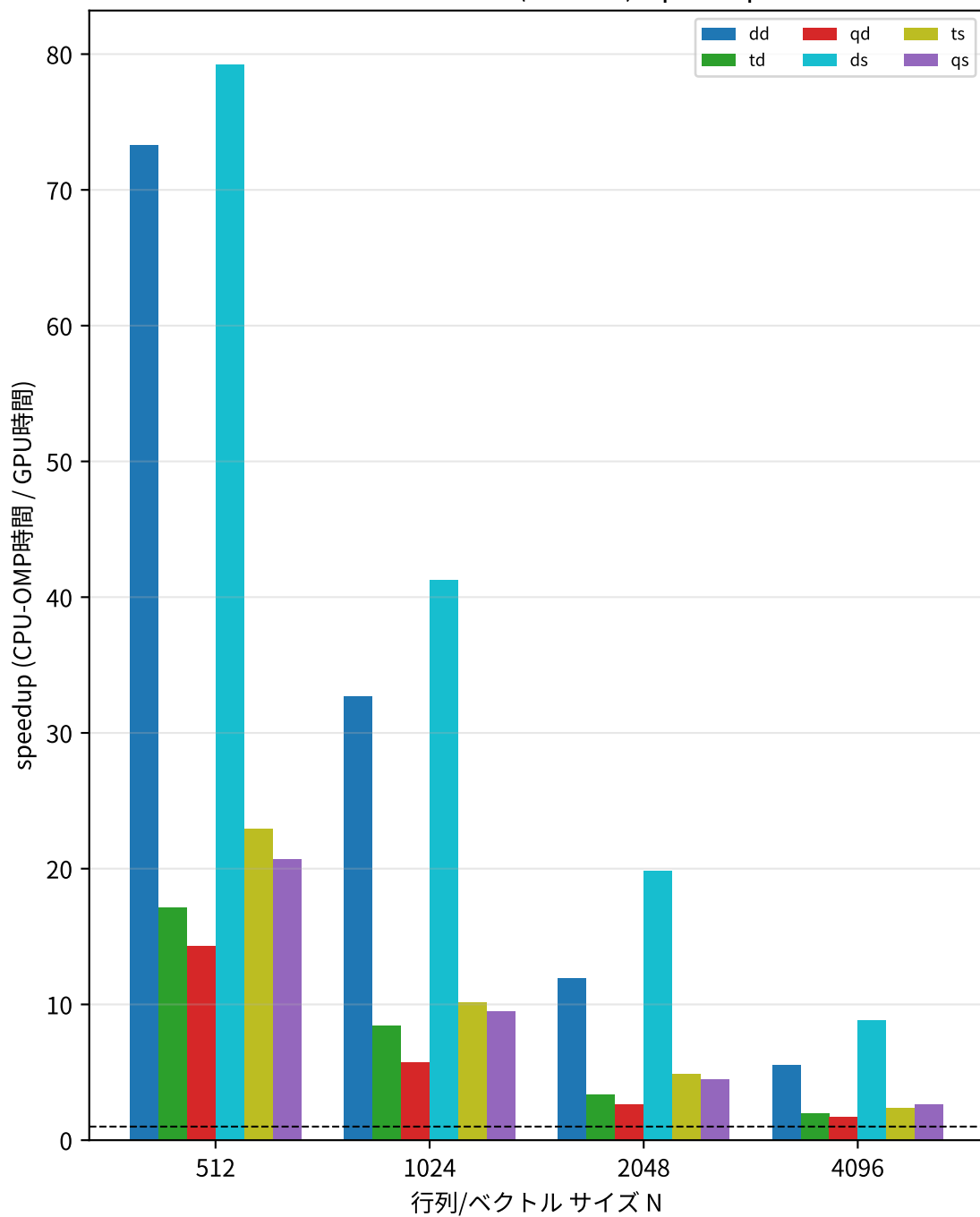
3. 対象精度

実数: dd(106bit) td(159) qd(212) / ds(48) ts(72) qs(96). 複素: cmpf 106/212/424 bit.

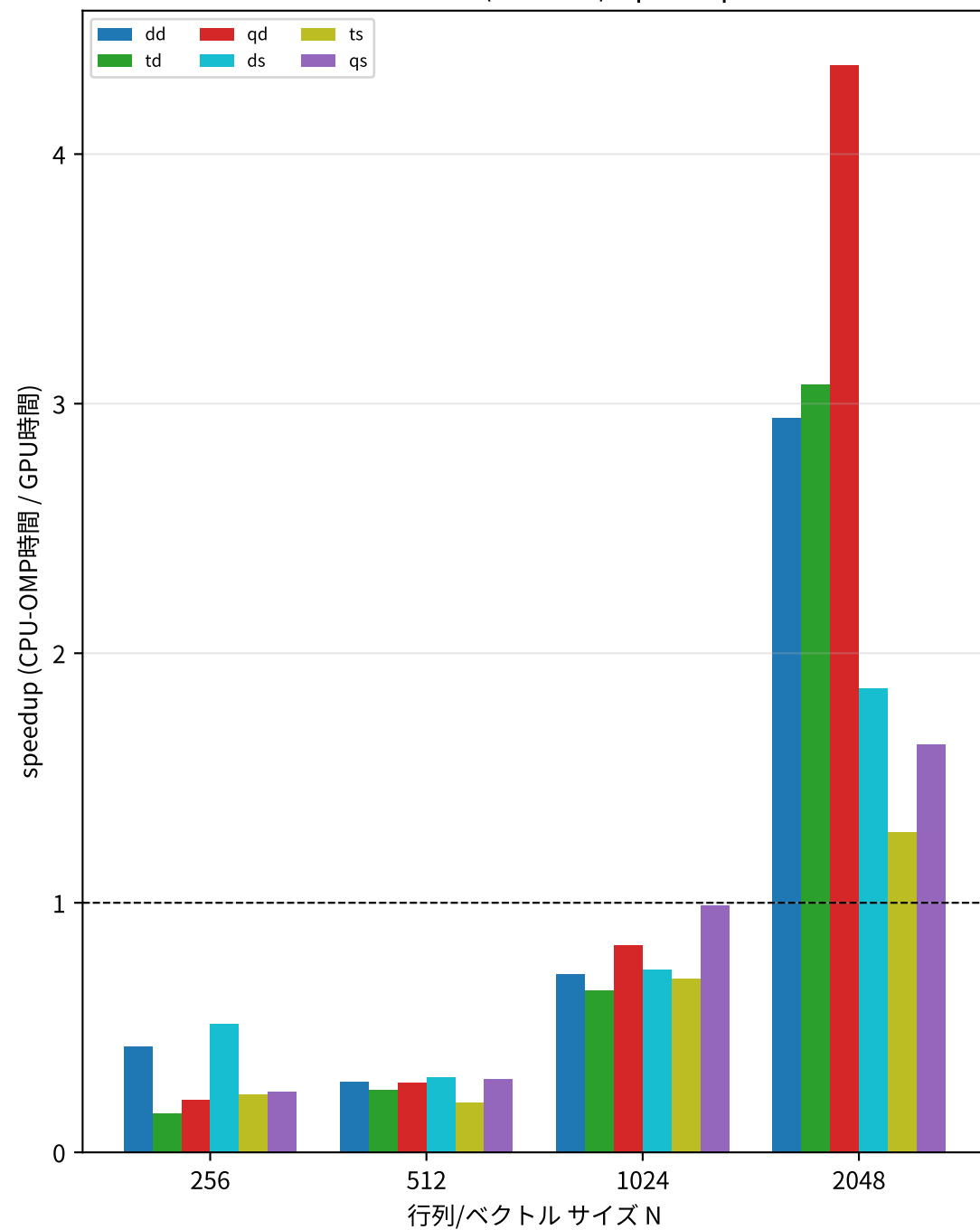
4. 収録シート/図: 概要, matvec, matmul, spmv, axpy, complex_mpc (xlsx) + 各演算の speedup 図(PDF).

実数 EFT 多倍長 (gdtq) : 密行列 基本線形計算 GPU vs CPU-OMP(32スレッド)

行列×ベクトル (matvec) speedup

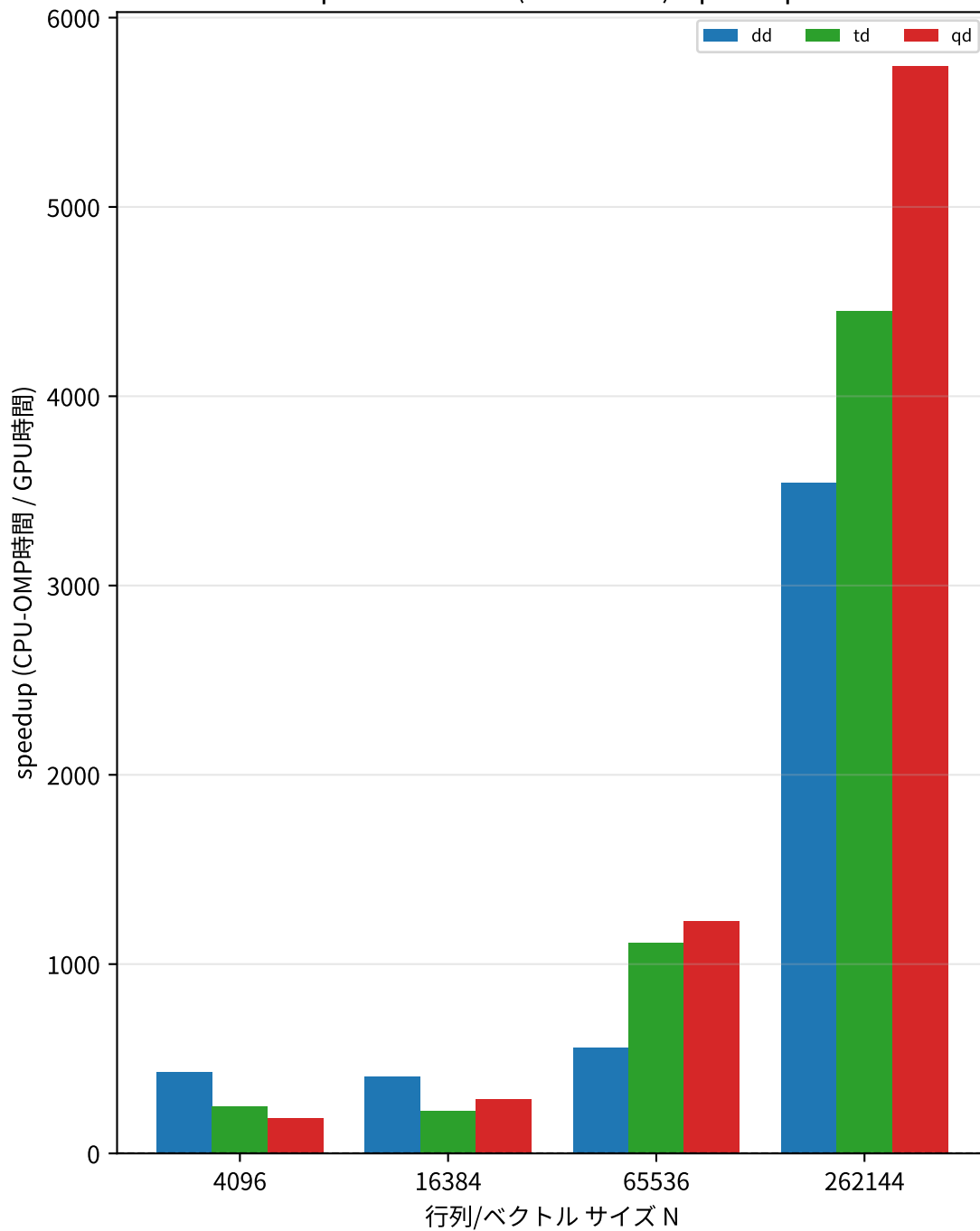


行列×行列 (matmul) speedup

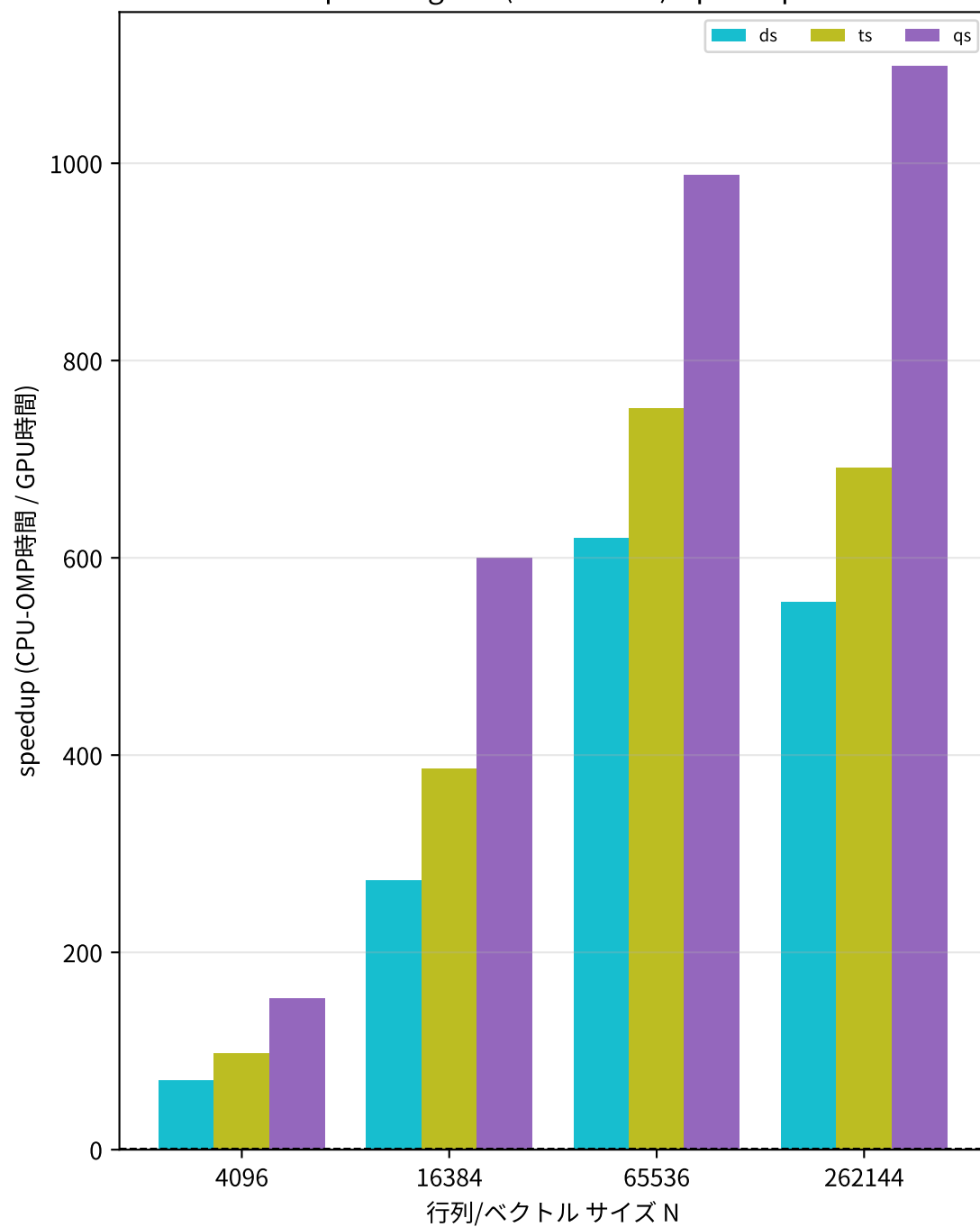


実数 EFT 多倍長 (gdtq) : 疎行列ベクトル積 SpMV GPU vs CPU

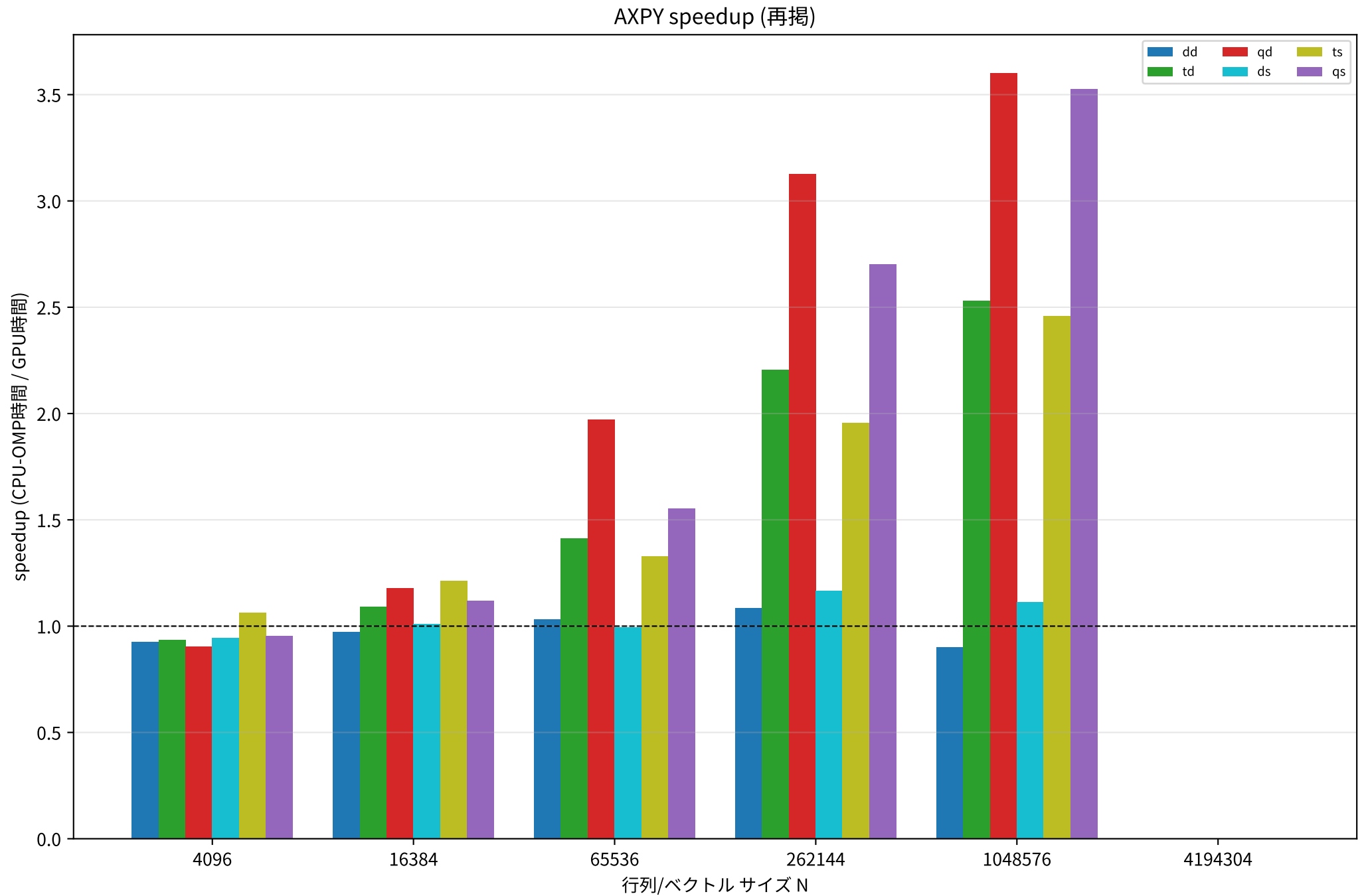
SpMV double系 (CPU=OMP) speedup



SpMV single系 (CPU=serial) speedup

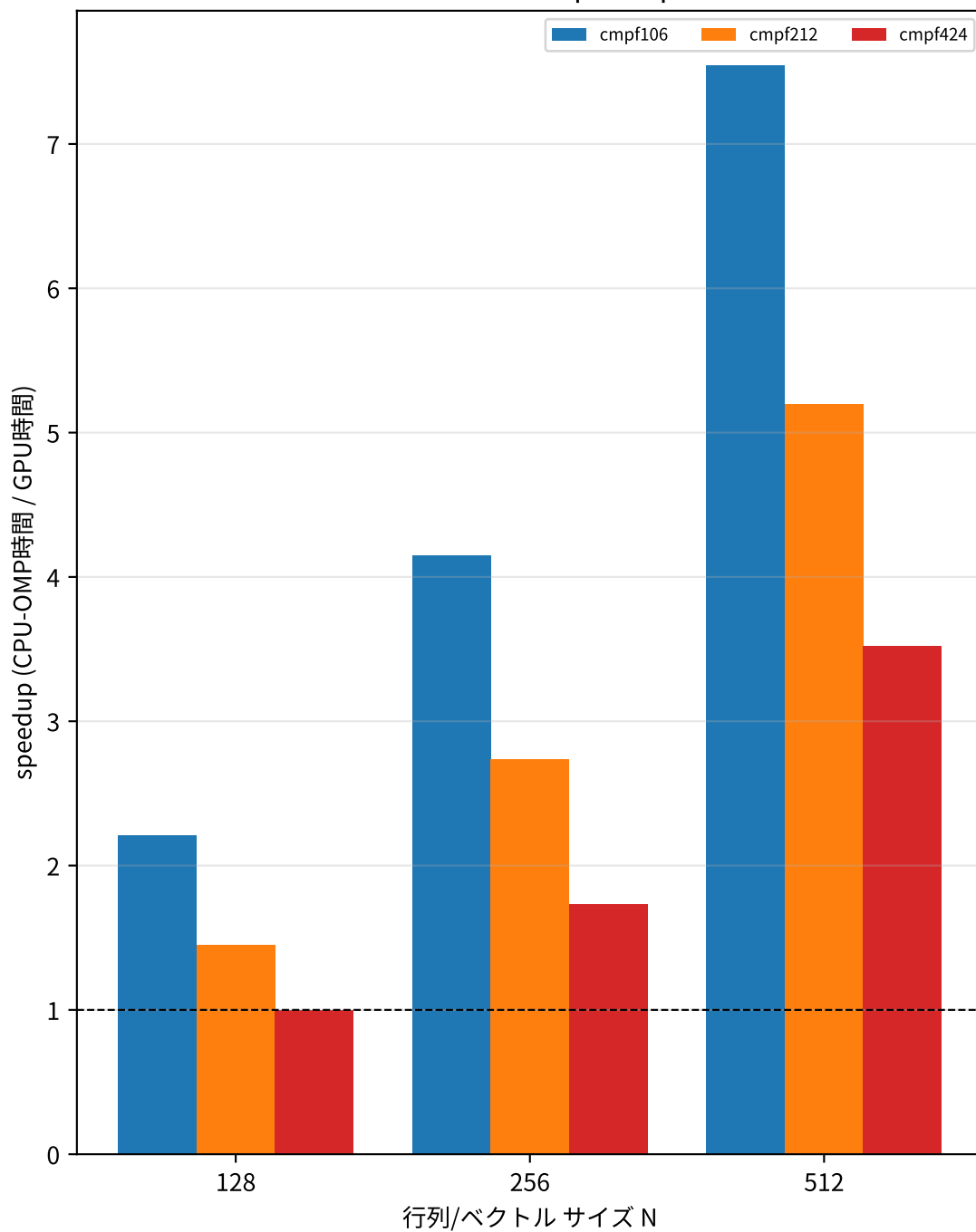


実数 AXPY ($c=a+\alpha*b$) GPU(最大スレッド) vs CPU-OMP(AVX-512,32スレッド)

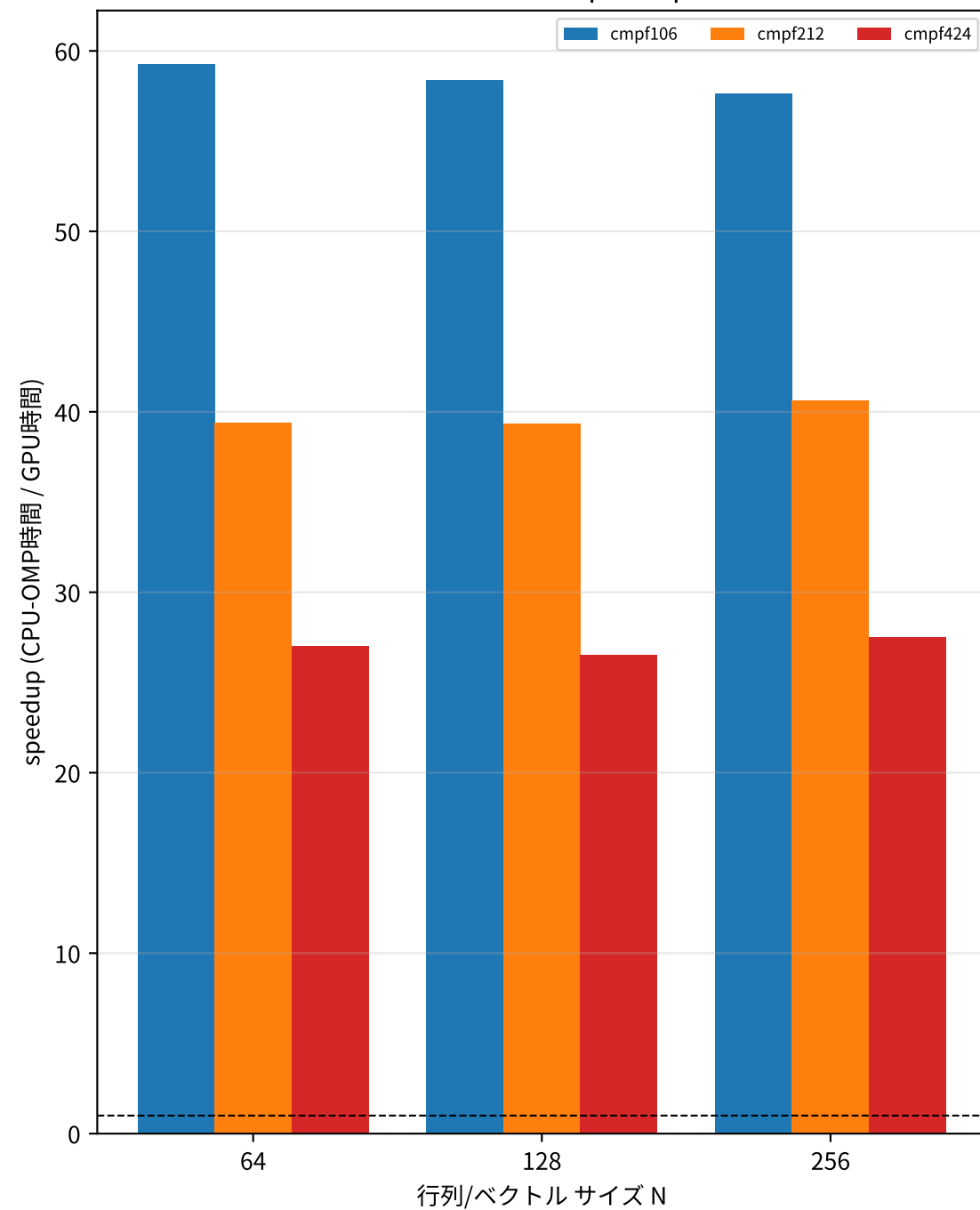


複素 多倍長 MPC (mpc_cuda) : GPU vs CPU-OMP(同一MPC演算の並列化)

複素 matvec speedup



複素 matmul speedup



まとめ: 各演算・各精度の代表点 (最大 N)

演算	型	bits	N	nnz	GPU[s]	CPU-OMP[s]	speedup	relerr
axpy	cmpf106	106	65536	0	0.0003327	0.08722	262.2x	0.0e+00
axpy	cmpf212	212	65536	0	0.000546	0.09523	174.4x	0.0e+00
axpy	cmpf424	424	65536	0	0.0008528	0.08532	100.0x	0.0e+00
axpy	dd	106	1048576	0	2.897e-05	2.606e-05	0.9x	0.0e+00
axpy	double	53	4194304	0	3.56e-05	0.0001423	4.0x	0.0e+00
axpy	ds	48	1048576	0	1.431e-05	1.594e-05	1.1x	0.0e+00
axpy	float	24	4194304	0	2.677e-05	2.32e-05	0.9x	0.0e+00
axpy	qd	212	1048576	0	5.266e-05	0.0001897	3.6x	0.0e+00
axpy	qs	96	1048576	0	2.773e-05	9.775e-05	3.5x	0.0e+00
axpy	td	159	1048576	0	3.72e-05	9.413e-05	2.5x	0.0e+00
axpy	ts	72	1048576	0	2.051e-05	5.044e-05	2.5x	0.0e+00
matvec	cmpf106	106	512	0	0.03608	0.2723	7.5x	0.0e+00
matvec	cmpf212	212	512	0	0.0558	0.2899	5.2x	0.0e+00
matvec	cmpf424	424	512	0	0.07359	0.2589	3.5x	0.0e+00
matvec	dd	106	4096	0	0.001273	0.006995	5.5x	-1.0e+00
matvec	ds	48	4096	0	0.0007391	0.006513	8.8x	-1.0e+00
matvec	qd	212	4096	0	0.005278	0.008975	1.7x	-1.0e+00
matvec	qs	96	4096	0	0.003071	0.008044	2.6x	-1.0e+00
matvec	td	159	4096	0	0.004002	0.007995	2.0x	-1.0e+00
matvec	ts	72	4096	0	0.002909	0.006941	2.4x	-1.0e+00
matmul	cmpf106	106	256	0	0.2922	16.84	57.6x	0.0e+00
matmul	cmpf212	212	256	0	0.4566	18.55	40.6x	0.0e+00
matmul	cmpf424	424	256	0	0.5994	16.5	27.5x	0.0e+00
matmul	dd	106	2048	0	1.952	5.744	2.9x	-1.0e+00
matmul	ds	48	2048	0	1.567	2.91	1.9x	-1.0e+00
matmul	qd	212	2048	0	5.595	24.38	4.4x	-1.0e+00
matmul	qs	96	2048	0	3.82	6.24	1.6x	-1.0e+00
matmul	td	159	2048	0	4.227	13.01	3.1x	-1.0e+00
matmul	ts	72	2048	0	2.935	3.766	1.3x	-1.0e+00
spmv	dd	106	262144	8126224	0.0001268	0.4492	3541.4x	0.0e+00
spmv	ds	48	262144	8126224	7.894e-05	0.04379	554.7x	2.0e-09
spmv	qd	212	262144	8126224	0.0002373	1.363	5742.2x	0.0e+00
spmv	qs	96	262144	8126224	0.0002004	0.2201	1098.4x	0.0e+00
spmv	td	159	262144	8126224	0.0001795	0.7989	4450.7x	0.0e+00
spmv	ts	72	262144	8126224	0.0001619	0.1119	690.9x	0.0e+00

注: speedup>1 は GPU が高速. 各 (演算,型) の最大 N の行を掲載. relerr は GPU と CPU の相対誤差最大値で正当性を示す(0 は下位ビットまで一致). SpMV ds/ts/qs の CPU は serial (ライブラリに OMP 版無し).

考察・結論

■ 正当性

全演算で GPU 結果は CPU と下位ビットまで一致($\text{relerr}=0$, 一部の単精度系のみ丸め由来 $\sim 1\text{e-}9 \sim 1\text{e-}12$). 多倍長 EFT / MPC 演算が GPU 上で正しく実装されていることを確認した.

■ 密行列 基本線形計算

- ・行列×ベクトル(matvec): GPU が最大 79倍高速. ただし CPU-OMP 側はスレッド生成オーバーヘッド($\sim 7\text{ms}$)律速で, 本質的にレイテンシ限界. GPU カーネルは μs オーダで完了.
- ・行列×行列(matmul): speedup 0.2 $\sim$ 4.4倍. ナイーブな GPU カーネル(1スレッド1要素, 共有メモリ・タイリング無し)は AVX-512+ブロック化+32スレッドの CPU-OMP に対し $N \leq 2048$ では優位でない. N の増加とともに比が改善(crossover 傾向)し, タイリング/Strassen 化が今後の課題.

■ 疎行列ベクトル積 SpMV

- ・GPU(CSR)が最大 5742倍高速. GPU は μs オーダ. ただし CPU-OMP 版は行オフセットの前置和を行毎に再計算する $O(N^2)$ 実装のため大 N で急速に悪化しており, 大 N の高速比はこの CPU 側非効率も含む. 小 $\sim$ 中 N では OMP 生成オーバーヘッド律速.

■ AXPY(再掲)

- ・GPU(全スレッド)対 CPU-OMP(32スレッド, AVX-512): 大 N で最大 4.0倍. 帯域律速演算で, 小 N は両者ともオーバーヘッド律速(比 ~ 1). 既測の結果を一貫指標で再掲.

■ 複素 多倍長 MPC(mpc_cuda)

- ・行列×ベクトル 最大 8倍, 行列×行列 最大 59倍(CPU は同一 MPC 演算の OpenMP 並列). 1要素あたりの演算コストが極めて大きい MPC では GPU の多数スレッドが非常に有効で, EFT の密行列積(ナイーブ GPU が不利)と対照的に, 高精度・高演算密度ほど GPU 優位が明確.

■ 総括

演算密度が高い(高精度・複素 MPC)ほど, また要素独立並列度が高い(SpMV, AXPY, matvec)ほど GPU 化の効果が大きい. EFT 密行列積のみ CPU-OMP が優位で, カーネル最適化が残課題.